



O que são Exceptions em Python?

Exceptions em Python são erros que ocorrem durante a execução de um programa, sendo usadas para sinalizar situações excepcionais que podem interromper o fluxo normal do código. A principal forma de tratar esses erros é utilizando os blocos try except.

Exemplos de Exceptions comuns

Vamos explorar alguns exemplos comuns de exceptions em Python:

ValueError: ocorre quando uma função recebe um argumento com o tipo certo, mas valor inapropriado.

```
x = int("hello")
```

Neste exemplo, tentamos converter a string "hello" para um inteiro, o que gera um ValueError.

ZeroDivisionError: ocorre quando tentamos dividir um número por zero.

```
resultado = 10 / 0
```

Aqui, tentamos dividir 10 por 0, gerando um ZeroDivisionError.

IndexError: ocorre quando tentamos acessar um índice que não existe em uma lista.

```
lista = [1, 2, 3]  
print(lista[5])
```

Neste caso, tentamos acessar o índice 5 de uma lista que só possui 3 elementos, gerando um IndexError.

O que são Syntax Errors em Python?

Syntax errors em Python são erros relacionados à estrutura do código que não segue as regras gramaticais da linguagem Python. Esses erros são detectados pelo interpretador antes mesmo do código ser executado.

Exemplos de Syntax Errors comuns

Aqui estão alguns exemplos comuns de syntax errors:

Erro de parênteses não fechados:

```
print("Hello"
```

Neste exemplo, esquecemos de fechar o parêntese, resultando em um SyntaxError.

Erro de indentação:

```
if True:  
print("Indentação incorreta")
```

Aqui, a linha `print("Indentação incorreta")` deveria estar indentada, gerando um IndentationError.



Erro de string não fechada:

```
print('Hello)
```

Neste caso, esquecemos de fechar a string com um apóstrofo, gerando um `SyntaxError`.



Lidando com Exceptions utilizando o bloco try except em Python

Lidar com erros é uma parte essencial do desenvolvimento de software. Em Python, uma das formas mais eficazes de tratar esses erros é utilizando o bloco try except.

Um exemplo simples do bloco try except em Python

Como funciona o bloco try except em Python?

O bloco try except permite que você “tente” executar um bloco de código e, caso ocorra um erro, “capture” essa exceção e execute um código alternativo. Vamos ver um exemplo simples:

```
try:
    x = int(input("Digite um número: "))
    y = int(input("Digite outro número: "))
    resultado = x / y
    print(f"O resultado é: {resultado}")
except ZeroDivisionError:
    print("Erro: Não é possível dividir por zero!")
except ValueError:
    print("Erro: Entrada inválida! Por favor, digite um número.")
```

Neste exemplo, o bloco try tenta executar a divisão de dois números fornecidos pelo usuário. Se o usuário tentar dividir por zero, o except ZeroDivisionError captura essa exceção e exibe uma mensagem de erro. Além disso, se o usuário digitar algo que não seja um número, o except ValueError captura essa exceção e exibe uma mensagem de erro apropriada.

Mostrando o erro ao usuário com Exception as

Como mostrar detalhes do erro ao usuário?

Para mostrar detalhes específicos do erro ao usuário, podemos usar a sintaxe except Exception as e. Isso nos permite capturar a exceção em uma variável e exibir informações detalhadas sobre o erro, por exemplo:

```
try:
    x = int(input("Digite um número: "))
    y = int(input("Digite outro número: "))
    resultado = x / y
    print(f"O resultado é: {resultado}")
except Exception as e:
    print(f"Ocorreu um erro: {e}")
```

Nesse caso, qualquer exceção que ocorra dentro do bloco try será capturada pelo except Exception as e, e a mensagem de erro será armazenada na variável e. Em seguida, exibimos essa mensagem ao usuário.



Mostrando o erro ao usuário com RuntimeError as

Como usar RuntimeError para capturar erros específicos?

O **RuntimeError** é uma exceção genérica que pode ser usada para capturar erros que ocorrem durante a execução do programa. Podemos usá-lo para capturar e exibir mensagens de erro específicas, por exemplo:

```
try:
    x = int(input("Digite um número: "))
    y = int(input("Digite outro número: "))
    if y == 0:
        raise RuntimeError("Divisão por zero não é permitida.")
    resultado = x / y
    print(f"O resultado é: {resultado}")
except RuntimeError as e:
    print(f"Erro de execução: {e}")
except ValueError:
    print("Erro: Entrada inválida! Por favor, digite um número.")
```

Aqui, usamos `raise RuntimeError` para lançar uma exceção personalizada quando o usuário tenta dividir por zero. O `except RuntimeError as e` captura essa exceção e exibe a mensagem de erro personalizada.

Utilizando o bloco try except else

Quando usar o bloco `else` com `try except`?

O bloco `else` é executado apenas se o bloco `try` não gerar nenhuma exceção. Portanto, isso é útil para separar o código que deve ser executado apenas quando não há erros. Exemplo:

```
try:
    x = int(input("Digite um número: "))
    y = int(input("Digite outro número: "))
    resultado = x / y
except ZeroDivisionError:
    print("Erro: Não é possível dividir por zero!")
except ValueError:
    print("Erro: Entrada inválida! Por favor, digite um número.")
else:
    print(f"O resultado é: {resultado}")
```

Acima, o bloco `else` contém o código que exibe o resultado da divisão. Ele só será executado se nenhuma exceção for lançada no bloco `try`.



Utilizando o bloco try except else finally

Como garantir a execução de um código, independentemente de erros?

O bloco finally é executado independentemente de qualquer exceção que ocorra. Isso é útil para liberar recursos ou executar ações de limpeza. Exemplo:

```
try:
    x = int(input("Digite um número: "))
    y = int(input("Digite outro número: "))
    resultado = x / y
except ZeroDivisionError:
    print("Erro: Não é possível dividir por zero!")
except ValueError:
    print("Erro: Entrada inválida! Por favor, digite um número.")
else:
    print(f"O resultado é: {resultado}")
finally:
    print("Execução finalizada.")
```

Veja que o bloco finally contém o código que será executado independentemente de qualquer exceção. Isso garante que a mensagem “Execução finalizada.” seja exibida, mesmo que ocorra um erro.

Gerando uma Exception

Lidar com erros é uma parte essencial da programação em Python e, utilizando blocos try except, podemos capturar e tratar exceções de maneira eficiente. A seguir, vamos explorar como criar exceções e como personalizá-las para atender às nossas necessidades específicas.



Como gerar uma Exception e parar a execução do código

Para gerar uma exceção e parar a execução do código, utilizamos a palavra-chave `raise`. Isso nos permite lançar uma exceção em qualquer ponto do nosso código, interrompendo sua execução imediatamente.

Exemplo prático

```
def dividir(a, b):  
    if b == 0:  
        raise ValueError("O divisor não pode ser zero!")  
    return a / b  
try:  
    resultado = dividir(10, 0)  
except ValueError as e:  
    print(f"Erro: {e}")
```

Aqui, definimos a função `dividir` para receber dois parâmetros, `a` e `b`. Dentro da função, realizamos uma verificação para garantir que o divisor `b` não seja zero; caso contrário, lançamos uma exceção do tipo `ValueError` com a mensagem "O divisor não pode ser zero!". Em seguida, utilizamos um bloco `try` para tentar executar a função `dividir` com os valores 10 e 0. Como o divisor é zero, a exceção é imediatamente capturada pelo bloco `except`, que trata a exceção imprimindo a mensagem de erro correspondente.



Por que usar raise?

Usar raise é útil para garantir que erros críticos sejam tratados imediatamente. Isso é especialmente importante em situações onde continuar a execução do código poderia levar a resultados incorretos ou danificar dados.

Como criar uma Exception personalizada em Python

Além de usar exceções embutidas, podemos criar nossas próprias exceções personalizadas. Isso é útil quando queremos fornecer mensagens de erro mais específicas ou quando estamos desenvolvendo uma biblioteca e queremos que os usuários lidem com erros de maneira específica.

Exemplo Prático

Vamos criar uma exceção personalizada chamada DivisaoPorZeroError:

```
class DivisaoPorZeroError(Exception):
    def __init__(self, mensagem):
        self.mensagem = mensagem
        super().__init__(self.mensagem)
def dividir(a, b):
    if b == 0:
        raise DivisaoPorZeroError("Não é possível dividir por zero!")
    return a / b
try:
    resultado = dividir(10, 0)
except DivisaoPorZeroError as e:
    print(f"Erro: {e.mensagem}")
```

Observe que criamos uma exceção personalizada chamada DivisaoPorZeroError, que é uma classe que herda de Exception. Dentro da classe, definimos o método __init__ para inicializar a mensagem de erro. Em seguida, definimos a função dividir para verificar se o divisor b é zero e, caso seja, lançar a exceção DivisaoPorZeroError com a mensagem “Não é possível dividir por zero!”. Posteriormente, usamos um bloco try para tentar executar a função dividir com os valores 10 e 0. Como o divisor é zero, o bloco except captura a exceção DivisaoPorZeroError e trata a exceção imprimindo a mensagem de erro correspondente.

Por que criar exceções personalizadas?

Criar exceções personalizadas permite que você forneça mensagens de erro mais claras e específicas, facilitando a depuração e o tratamento de erros. Isso é especialmente útil em projetos maiores ou em bibliotecas que serão usadas por outras pessoas.

Boas Práticas

- Especificidade: crie exceções personalizadas para situações específicas que não são cobertas pelas exceções embutidas.
- Clareza: forneça mensagens de erro claras e informativas.
- Documentação: documente suas exceções personalizadas para que outros desenvolvedores saibam como usá-las e tratá-las.



Utilizando try except para lidar com erros comuns em Python

Agora que entendemos os conceitos básicos de try e except, vamos explorar como tratar erros comuns em Python usando essa poderosa ferramenta.

Arquivo não encontrado: FileNotFoundError

Quando tentamos abrir um arquivo que não existe, o Python lança um FileNotFoundError. Então, para evitar que nosso programa quebre, podemos usar try except para tratar esse erro.

```
Try:
    f = open('arquivo_inexistente.txt', 'r')
except FileNotFoundError:
    print("Erro: Arquivo não encontrado!")
```

Neste exemplo, tentamos abrir um arquivo chamado arquivo_inexistente.txt para leitura. Se o arquivo não for encontrado, o bloco except é executado, exibindo uma mensagem de erro. Dessa forma, evita-se que o programa pare abruptamente.

Transformando string em inteiro: ValueError

Quando tentamos converter uma string que não representa um número em um inteiro, o Python lança um ValueError. Podemos usar try except python para tratar esse erro e fornecer uma mensagem mais amigável ao usuário.

```
try:
    numero = int(input("Digite um número: "))
except ValueError:
    print("Erro: Você não digitou um número válido!")
```

Aqui, pedimos ao usuário para digitar um número. Se o usuário digitar algo que não pode ser convertido em um inteiro, como "abc", o bloco except é executado, informando ao usuário que a entrada não é válida.

Buscando chave inexistente em um dicionário: KeyError

Quando tentamos acessar uma chave que não existe em um dicionário, o Python lança um KeyError. Para que evitemos que o programa quebre, podemos usar try except para tratar esse erro.

```
capitais = {'Brasil': 'Brasília', 'França': 'Paris'}
try:
    print(capitais['Inglaterra'])
except KeyError:
    print("Erro: Chave não encontrada no dicionário!")
```

Neste exemplo, tentamos acessar a chave 'Inglaterra' em um dicionário de capitais. Mas, como essa chave não existe, o bloco except é executado, exibindo uma mensagem de erro.



Somando uma string e um número: TypeError

Quando tentamos somar uma string e um número, o Python lança um `TypeError`. Podemos usar `try except` para tratar esse erro e fornecer uma mensagem mais clara.

```
try:
    resultado = 'abc' + 123
except TypeError:
    print("Erro: Não é possível somar uma string e um número!")
```

Aqui, tentamos somar a string 'abc' com o número 123, mas essa operação não é permitida. Dessa forma, o bloco `except` é executado, informando ao usuário que a soma não é possível.

Dividindo valor por zero: ZeroDivisionError

Quando tentamos dividir um número por zero, o Python lança um `ZeroDivisionError`, mas também podemos usar `try except` para tratá-lo e evitar que o programa quebre.

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Erro: Não é possível dividir por zero!")
```

Nesse caso, tentamos dividir 10 por 0, no entanto essa operação não é permitida. Portanto, o bloco `except` é executado, exibindo uma mensagem de erro.

Importando biblioteca não instalada: ModuleNotFoundError

Quando tentamos importar uma biblioteca que não está instalada, o Python lança um `ModuleNotFoundError`. Então, podemos usar `try except` para tratar esse erro e fornecer uma mensagem mais clara.

```
try:
    import biblioteca_inexistente
except ModuleNotFoundError:
    print("Erro: Biblioteca não encontrada!")
```

Aqui, tentamos importar uma biblioteca chamada `biblioteca_inexistente`, que não está instalada. Assim, o bloco `except` é executado, informando ao usuário que a biblioteca não foi encontrada.